
Hydra-Nav: Object Navigation via Adaptive Dual-Process Reasoning

Zixuan Wang^{1,2} Huang Fang¹ Shaoan Wang^{1,3} Yuanfei Luo¹ Heng Dong¹ Wei Li^{†1} Yiming Gan^{†4}

Abstract

While large vision-language models (VLMs) show promise for object goal navigation, current methods still struggle with low success rates and inefficient localization of unseen objects—failures primarily attributed to weak temporal-spatial reasoning. Meanwhile, recent attempts to inject reasoning into VLM-based agents improve success rates but incur substantial computational overhead. To address both the ineffectiveness and inefficiency of existing approaches, we introduce HYDRA-NAV, a unified VLM architecture that adaptively switches between a deliberative “slow system” for analyzing exploration history and formulating high-level plans, and a reactive “fast system” for efficient execution. We train HYDRA-NAV through a three-stage curriculum: (i) spatial-action alignment to strengthen trajectory planning, (ii) memory-reasoning integration to enhance temporal-spatial reasoning over long-horizon exploration, and (iii) iterative rejection fine-tuning to enable selective reasoning at critical decision points. Extensive experiments demonstrate that HYDRA-NAV achieves state-of-the-art performance on the HM3D, MP3D, and OVON benchmarks, outperforming the second-best methods by 11.1%, 17.4%, and 21.2%, respectively. Furthermore, we introduce SOT (Success weighted by Operation Time), a new metric to measure search efficiency across VLMs with varying reasoning intensity. Results show that adaptive reasoning significantly enhances search efficiency over fixed-frequency baselines. Project page is available at: <https://zixuan-wang99.github.io/Hydra-Nav/>.

¹ByteDance Seed ²Institute of Automation, Chinese Academy of Sciences (CASIA) ³Peking University ⁴Institute of Computing Technology, Chinese Academy of Sciences (ICT). Correspondence to: Wei Li <liwei.85@bytedance.com>, Yiming Gan <ganyiming@ict.ac.cn>.

1. Introduction

Object goal navigation is an emerging embodied task that requires a robot to actively explore the physical world and locate target objects using only egocentric perception (Chaplot et al., 2020b; Sun et al., 2024). Object navigation has attracted considerable attention from both academia and industry, as the ability to search for objects is essential for deploying robots in home environments. This task demands a combination of multiple core capabilities: (i) strong spatial reasoning to understand the spatial structure of the environment; (ii) effective long-term memory to retain information about explored scenes; (iii) robust long-horizon trajectory planning to explore the environment (Habibpour & Afghah, 2025; Liu et al., 2025b).

The rise of large vision-language models (VLMs) has significantly advanced object navigation. Recent works (Yu et al., 2023; Nasiriany et al., 2024; Yokoyama et al., 2024a; Goetting et al., 2024; Zhong et al., 2025a; Zhang et al., 2025a) leverage VLMs to improve navigation success rates, benefiting from their strong generalization capabilities. Despite these advances, current methods still fall short of human-level performance. This gap primarily stems from the limited temporal-spatial understanding ability of the underlying VLMs. For instance, the model often fails to maintain a coherent memory of visited regions, leading to redundant exploration of the same areas. To narrow the gap, recent works incorporate memory mechanisms (Zhang et al., 2024b; Yang et al., 2025; Zhang et al., 2024a) and explicit reasoning capabilities (Nie et al., 2025; Zhu et al., 2025; Cao et al., 2025; Li et al., 2025; Cai et al., 2025; Qiao et al., 2025; Liu et al., 2025a; Xiang et al., 2025) at each inference step. However, their reasoning process tends to focus on current observations, overlooking the analysis of historical trajectory and progress of exploration. Furthermore, while frequent reasoning improves success rates, it leads to unnecessary inference costs during routine tasks. For instance, walking down a long, unobstructed hallway yields highly consistent visual inputs, which are best handled by simple reactive control.

We introduce HYDRA-NAV, a VLM tailored for object navigation. To improve the temporal-spatial reasoning ability of the base VLM, we develop a data synthesis pipeline and collect 500K trajectories with high-quality reasoning from simulated environments. We carefully design the rea-

soning synthesis prompt to include three components: reviewing historical memory, analyzing current observation, and planning future trajectory based on both. The base VLM (Qwen2.5-VL-7B) trained on our data demonstrates strong temporal-spatial reasoning ability and achieves state-of-the-art performance on widely-used object navigation benchmarks. To reduce the overhead of frequent reasoning, we design HYDRA-NAV as a slow-fast system that allows triggering reasoning at arbitrary frequency. Meanwhile, we use iterative rejection-sampling finetuning (IRFT) to teach HYDRA-NAV to adaptively switch between the slow and fast modes. IRFT not only increases search efficiency but also boosts the success rate across all settings. Our contributions are summarized as follows:

- We propose HYDRA-NAV, a navigation agent that supports adaptive temporal-spatial reasoning within a single VLM architecture. Unlike prior approaches that trigger reasoning at a fixed frequency, HYDRA-NAV learns to adaptively switch between a strategic slow system and a reactive fast system, reducing reasoning overhead while maintaining a high success rate.
- We design a curriculum training pipeline to progressively enhance HYDRA-NAV’s spatial-action alignment (Section 3.3), temporal-spatial reasoning (Section 3.4), and adaptive reasoning ability (Section 3.5).
- We conduct extensive experiments on widely-used object navigation benchmarks (HM3D, MP3D, and OVON). HYDRA-NAV-SFT with fixed reasoning frequency achieves a 50.9% success rate on the OVON Val-Unseen benchmark, 5.7% higher than the previous state-of-the-art. HYDRA-NAV-IRFT achieves state-of-the-art success rate on HM3D, MP3D, and OVON, outperforming the second-best method by 11.1%, 17.4%, and 22.3% respectively. Furthermore, we introduce a new metric, SOT (Success weighted by Operation Time), to evaluate the search efficiency of navigation agents with different reasoning intensity. We show that HYDRA-NAV-IRFT achieves better SOT than HYDRA-NAV-SFT, demonstrating the effectiveness of IRFT.

Conflict of Interest Disclosure. Work partially completed during an author’s internship at ByteDance. The authors declare no conflicts of interest or product endorsements.

2. Related Work

2.1. Vision Language Models for Object Navigation

Incorporating vision language models (VLMs) into object navigation has shifted the field from traditional methods—ranging from end-to-end reinforcement learning policies (Zhu et al., 2017; Yang et al., 2018; Lyu et al., 2022;

Druon et al., 2020; Wijmans et al., 2019) to modular map-based planning (Chaplot et al., 2020b;a; Ramakrishnan et al., 2022; Chaplot et al., 2020c)—toward leveraging pre-trained knowledge. VLMs-based methods enable agents to interpret high-level instructions and perform common sense reasoning, facilitating robust open-world understanding and generalization.

VLM-based navigation agent generally follows three paradigms. First, modular agent system approaches (Yu et al., 2023; Nasiriany et al., 2024; Yokoyama et al., 2024a; Goetting et al., 2024; Zhou et al., 2025b; Yang et al., 2025; Wang et al., 2026) adopt a hierarchical framework where the VLM acts as a global policy to select waypoints, leaving execution to a deterministic local planner. Recent works (Nie et al., 2025; Zhang et al., 2025c; Long et al., 2024; Zhu et al., 2025; Cao et al., 2025; Li et al., 2025) further enhance this by using Chain-of-Thought (CoT) for deeper scene analysis and planning. Second, end-to-end methods (Zhang et al., 2024a; Gao et al., 2025; Zhang et al., 2024b; Zhong et al., 2025a; Zhang et al., 2025a; Liu et al., 2025a; Xiang et al., 2025) fine-tune VLMs to directly predict low-level control actions from visual inputs. Third, dual-system architectures (Wei et al., 2025; Xue et al., 2025) employ a “slow-fast” paradigm, where a VLM generates periodic sub-goals to guide a lightweight, high-frequency controller.

However, a critical trade-off between reasoning depth, inference efficiency, and system unity persists across these paradigms. Modular agent system and end-to-end approaches incur prohibitive latency by applying CoT reasoning at every control step. While dual system architectures mitigate this via hierarchical “slow-fast” paradigm, they suffer from architectural fragmentation and rigid, heuristic switching, lacking the flexibility to adaptively reasoning. In contrast, HYDRA-NAV unifies the dual-process mechanism within a single VLM, enabling adaptive switching between direct low-level execution and CoT reasoning.

2.2. Adaptive Reasoning

Adaptive thinking dynamically selects between direct response and step-by-step CoT thinking (Chen & Li, 2024; Chung et al., 2025; Wang et al., 2025a; Wu et al., 2025a;b; Yu et al., 2025; Zhan et al., 2025; Zhang et al., 2025b). Existing methods (Peng et al., 2025; Luo et al., 2025) typically rely on rule-based heuristics to categorize scene complexity, employing supervised fine-tuning or reinforcement learning to learn when to think. Besides, Game-tars (Wang et al., 2025b) utilizes rejection fine-tuning on extensive human-annotated datasets to mimic human-like adaptive reasoning. However, in the domain of object navigation, manually demarcating simple versus challenging scenarios is non-trivial, and large-scale human demonstrations of adaptive reasoning are scarce. Crucially, unlike recent dual-

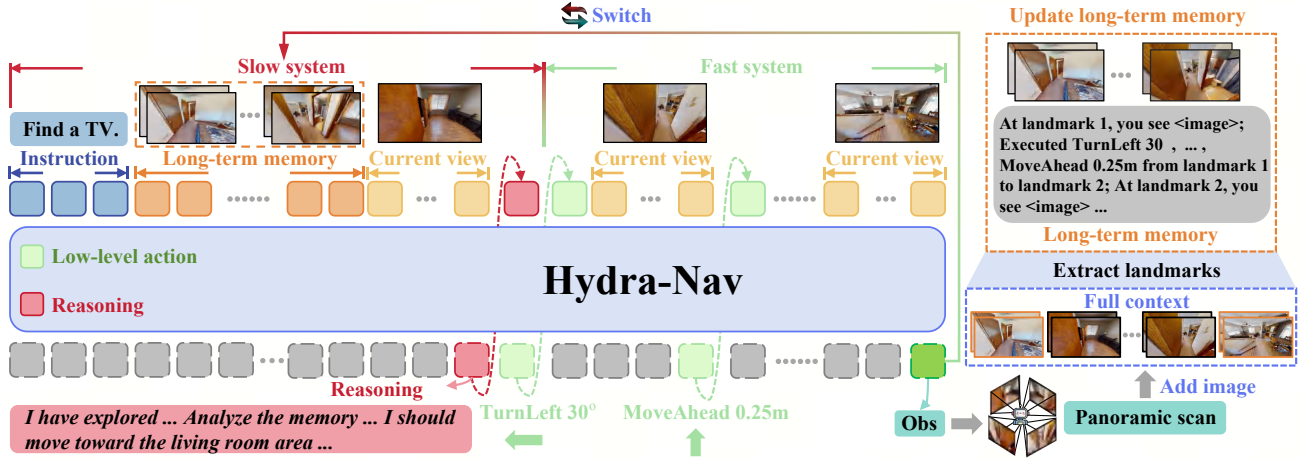


Figure 1. The architecture of HYDRA-NAV. HYDRA-NAV receives user instruction, long-term memory, and previous image-action pairs, then outputs reasoning (optionally) and meta-actions. HYDRA-NAV adaptively switches between the fast and slow systems by outputting the special transition token `obs`.

process approaches that rely on modular pipelines (Zhong et al., 2025b; Yue et al., 2025; Zhou et al., 2025a), HYDRA-NAV pioneers the unification of adaptive slow-fast reasoning within a single VLM architecture for object navigation. To address this, we introduce iterative rejection fine-tuning (IRFT). Instead of relying on external supervision, we leverage on-policy rollouts to autonomously finding “stagnation points”—moments where reactive low-level action policies fail. By injecting reasoning retrospectively at these points, we train the model to trigger CoT adaptively when necessary.

3. Method

3.1. Problem Formulation

We formulate object goal navigation as a partially observable markov decision process (POMDP). At each time step t , the agent receives an egocentric observation o_t and a natural language instruction I_{goal} specifying the target object. Based on the history H_t , the agent generates an output y_t from a hybrid action space $\mathcal{A} = \mathcal{A}_{motor} \cup \mathcal{A}_{reason}$. The motor action space $\mathcal{A}_{motor}$ comprises discrete locomotion primitives (e.g., `MoveAhead 0.25m`) and a specialized system transition token `obs` to decide if switch to the slow system. The reasoning space $\mathcal{A}_{reason}$ consists of natural language tokens used by the slow system to generate the reasoning text. This unified formulation allows a VLM to flexibly alternate between executing low-level control and generating high-level strategic guidance.

3.2. The Dual-process System

As illustrated in Figure 1, HYDRA-NAV unifies high-level planning and low-level meta actions within a single VLM architecture. Unlike prior hierarchical approaches that rely on separate models (Ding et al., 2025; Nie et al., 2025),

HYDRA-NAV operates as an end-to-end dual-process system. Below, we detail a full cycle of the dual process; see Figure 2 for an illustration.

Slow system. The cycle initiates with the slow system, responsible for global localization and high-level temporal-spatial reasoning. The input consists of the goal instruction (e.g., “Find a TV.”), the current panoramic observation comprising 4 RGB images obtained by rotating the head camera at 90° intervals ($0^\circ, 90^\circ, 180^\circ, 270^\circ$), and a structured long-term memory. We construct the memory as a serialized graph of text-image interleaved landmark nodes connected by action edges (see Appendix H). The model generates a reasoning text to analyze exploration progress followed by an immediate subsequent meta action (e.g., `<think_start> ...move toward the living room area ...<think_end>MoveAhead 0.25m`).

Fast system. Upon generating the high-level plan, the model transitions to the fast system. This phase operates as a multi-turn dialogue with long-term memory and reasoning text from the previous slow system as the system prompt. For subsequent steps, the model takes the dialogue history and the current observation frame as input. Utilizing KV-caching, the model encodes only the latest egocentric frame and autoregressively decodes low-level atomic actions (e.g., `MoveAhead 0.25m`). This design avoids reprocessing the full history context, thereby saving computation.

Adaptive switching mechanism. The transition from the fast system back to the slow system is self-triggered. When the agent completes a sub-goal in the high-level plan, or when the current observation invalidates the existing plan, the model outputs a special token `obs`. This token initiates a panoramic scan (four 90° rotations) to capture new observations and construct a new landmark node. The node is then connected to the previous landmark through the executed

action sequence and added to the long-term memory. To keep the context length manageable, the memory is dynamically pruned: the start and end nodes are always preserved, while intermediate landmarks are uniformly sampled, with a maximum of 10 landmarks retained. With the updated memory, the agent re-enters the slow system to revise its high-level plan.

Next, we describe our curriculum training pipeline that progressively enhance the spatial-action alignment, temporal-spatial reasoning and adaptive reasoning of HYDRA-NAV.

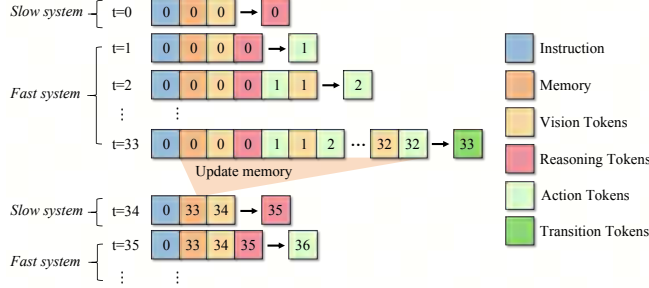


Figure 2. Illustration of the context organization of HYDRA-NAV during inference. The context starts with a system prompt containing the user instruction and long-term memory. Short-term memory is organized as interleaved image-action pairs. When a transition token is encountered, we update the memory and clear the image-action pairs. Note that each token block in the figure represents a sequence of multiple tokens.

3.3. Stage 1: Spatial-action Alignment

The goal of this stage is to collect a large number of trajectories to train the base VLM into a navigation agent. Following recent works (Zhang et al., 2024a; Liu et al., 2025a), we use annotated data from the training sets of HM3D (Yadav et al., 2023), MP3D (Chang et al., 2017), and OVON (Yokoyama et al., 2024b) to generate 500K trajectories using the A^* planner. Each trajectory is formatted as a multi-turn conversation:

$$\tau := \left\{ I_{\text{sys}}^{(1)}, (v_1, a_1), (v_2, a_2), \dots, (v_T, a_T) \right\},$$

where $I_{\text{sys}}^{(1)}$ represents the system instruction used in this stage, v_t denotes the egocentric RGB observation at time step t , $a_t \in \{\text{MoveAhead } 0.25\text{m}, \text{TurnLeft } 30^\circ, \text{TurnRight } 30^\circ, \text{End}\}$ is the meta action executed at time t . To reduce computational cost, we compute the loss over all turns in a single forward-backward pass, allowing each trajectory to be processed in one operation. The training parameters used by this stage is shown in Table 4.

3.4. Stage 2: Reasoning-memory Integration

After Stage 1, the base VLM is able to generate reasonable, collision-free trajectories in the simulated environment, but lacks the ability to explore the scene when its initial attempt

fails. Stage 2 equips the model with the ability to (i) explore the environment while recovering from failures, and (ii) reason on the long-term memory and plan for future exploration accordingly. Next, we describe how we enable the model to acquire these two capabilities.

Trajectory generation. Unlike Stage 1, which uses A^* to generate trajectories, here we construct trajectories using a heuristic waypoint selection algorithm. Specifically, for each scene, we define a score function $S(p; p_{\text{init}})$ over all reachable points $p \in \mathbb{R}^3$, conditioned on the robot’s initial position p_{init} . The score function balances spatial openness with proximity to the target goal, ensuring trajectories capture exploratory behavior (see Appendix A for details). For each initial position, we select the two highest-scoring points: $p_1, p_2 = \text{top-2}_{p \in \mathbb{R}^3} S(p; p_{\text{init}})$. The trajectory is then constructed as $p_{\text{init}} \rightarrow p_1 \rightarrow p_2 \rightarrow p_{\text{target}}$ or $p_{\text{init}} \rightarrow p_2 \rightarrow p_1 \rightarrow p_{\text{target}}$, depending on the total length of the path, where p_{init} denotes the starting position and p_{target} represents the location of the target object. As shown in Figure 8, the robot first visits these two high-scoring exploration points (Point 1 and Point 2) before finally reaching the target (e.g., a television).

Temporal-spatial reasoning synthesis. After generating trajectories with exploration described above, we partition each trajectory into a set of segments with fixed length (we set the length to be 16). Then we insert memory and reasoning text at the beginning of each segment, we also insert the transition token `obs` at the end of each segment. Each segment is formatted as a multi-turn conversation:

$$\begin{aligned} s_1 &:= \{ I_{\text{sys}}^{(2)}, (m_1, v_1, r_1, a_1), (v_2, a_2), \dots, (v_{16}, \text{obs}) \} \\ s_2 &:= \{ I_{\text{sys}}^{(2)}, (m_2, v_{17}, r_2, a_{17}), (v_{18}, a_{18}), \dots, (v_{32}, \text{obs}) \} \\ &\dots \end{aligned}$$

where s_i denote the i -th segment of the trajectory, m_i is the long-term memory (a compression of previous segments, see Appendix H for the details), r_i is the reasoning text of the i -th segment.

We synthesize reasoning text with the following procedure: at time t that triggers the slow system, we first prompt Qwen3-VL-235B Thinking (Bai et al., 2025) to review historical images and summarize the search progress so far. Then, we feed the summary, together with the current view and the future view (the panoramic scan at the beginning of the next segment) to the model. We ask it to produce reasoning text that serves to guide the future search plan for the target object. To prevent information leakage, we explicitly prompt the model to treat the future view only as a hidden guide, ensuring it is not mentioned in the output. Furthermore, we use a LLM to verify and filter out any reasoning text that inadvertently reveal future information. The prompt used to synthesize reasoning is shown in Appendix I. See the right side of Figure 3 for an illustration of

1) Path Data Generation



2) COT Data Synthesis

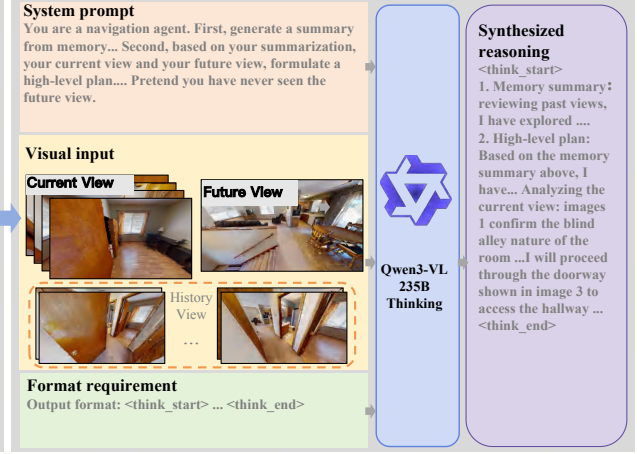


Figure 3. An illustration of the data synthesis pipeline used in stage 2. The left side shows our trajectory generation strategy, where the robot visits Point 1 and Point 2 before reaching the goal. The right side illustrates how we prompt Qwen3-VL-235B-Thinking to produce high-quality reasoning traces.

the reasoning synthesis workflow.

After reasoning synthesis, we can no longer process a full trajectory within one forward-backward pass, because different segments from the same trajectory do not share the same context. As a result, we train the model at the segment level, and the training efficiency of stage 2 is lower than that of stage 1. In addition to the navigation data, we also mix some visual question answering data into stage 2 training to maintain the general capability of the base VLM.

3.5. Stage 3: Adaptive Reasoning via Iterative Rejection Fine-tuning

While stage 2 establishes the fundamental mechanics of the dual-system architecture, it switches between the slow and fast systems at a fixed frequency. This static switching policy can incur unnecessary computational overhead in trivial scenarios and often fails to trigger the slow system at critical "stagnation points" where navigation collapses.

We first define "stagnation points" as moments where the agent fails to produce effective exploration progress. We identify two specific types of "stagnation points":

1. Repetitive exploration: The agent exhibits repetitive exploration at time t if it revisits a position from at least T_{stag} steps earlier, i.e., $\exists k \in [0, t - T_{\text{stag}}]$ such that $\|p_t - p_k\| \leq \delta_{\text{stag}}$.
2. Lack of progress: The agent lacks progress at time t if $\text{dist}(p_t, G) > \text{dist}(p_{t-\Delta t}, G)$, where $\Delta t \sim \mathcal{U}(20, 35)$. We choose the distribution $\mathcal{U}(20, 35)$ for two reasons: (i) uniform sampling increases data diversity and mitigates overfitting risk; (ii) the range $[20, 35]$ is large enough to avoid false positives when the agent navigates around obstacles.

We set $T_{\text{stag}} = 20$ and $\delta_{\text{stag}} = 0.5\text{m}$. These conditions determine when the reactive fast system requires intervention from the reasoning (slow) system.

To collect training data, we conduct on-policy rollouts where the agent operates primarily in the fast system and triggers the slow system at the defined "stagnation points", see Algorithm 1 for details. We retain trajectories where the agent successfully reaches the goal. Otherwise, we repair failed trajectories using different strategies based on their failure modes.

We observe that failed trajectories fall into one of two categories: (i) **Timeout**: The episode length exceeds the maximum step limit ($T > 200$); or (ii) **Target misidentification**: The agent misidentifies the target at time t if $a_t = \text{End}$ and $\text{dist}(p_t, G) > \delta_{\text{success}}$, where $\text{dist}(p_t, G)$ denotes the minimum distance between p_t and the target object set G .

For each failure mode, we first determine the intervention timestamp t^* : for target misidentification, we set t^* to the final step; for timeout, we scan the trajectory for stagnation points and select the point where the agent was spatially closest to the target.

Once the intervention timestamp t^* is identified, we use the following procedure to repair the trajectory:

- We substitute the erroneous action a_{t^*} with the transition token `obs`, thereby switching the model to the slow system at t^* .
- We construct a new trajectory by concatenating the history $p_{1:t^*}$ with the optimal path (generated via an A^* planner) from p_{t^*} to the target p_{target} .
- We keep the reasoning of the historical path $p_{1:t^*}$ and synthesize a new reasoning text for the corrective path

$p_{t^*} \rightarrow p_{\text{target}}$ following the similar procedure described in stage 2.

- If the length of the new trajectory exceeds 400, we simply drop this trajectory.

The detailed data collection procedure is described in Algorithm 2. The trajectories produced by this procedure do not trigger reasoning at a fixed frequency, allowing the model trained on them to adaptively switch to the slow system. We iteratively apply this "reject-and-repair" data synthesis strategy using the latest checkpoint, progressively refining the model's adaptive reasoning capability. This pipeline is analogous to the rejection fine-tuning methods used in LLM training (Touvron et al., 2023; Guo et al., 2025), and we refer to the model at this stage as HYDRA-NAV-IRFT.

3.6. Training Configurations

See Table 4 in the appendix for the training hyperparameters used in each stage.

4. Experiments

To evaluate the performance of HYDRA-NAV, we carry out extensive experiments on commonly used benchmarks. Specifically, we aim to address the following three research questions:

- Does HYDRA-NAV outperform existing methods in terms of success rate and search efficiency on unseen objects and scenes? How does HYDRA-NAV behave at different training stages?
- How do different training strategies (e.g., with/without reasoning and memory modules) and data collection strategies affect the final performance of HYDRA-NAV?
- How does IRFT influence the final success rate and task completion time—which includes both robot execution time and model reasoning time?

4.1. Experimental Setup

Benchmarks. We evaluate HYDRA-NAV on the HM3D (Yadav et al., 2023), MP3D (Chang et al., 2017) for object goal navigation, and OVON (Yokoyama et al., 2024b) for open-vocabulary object navigation.

Evaluation metrics. We report two widely-used metrics: success rate (SR) and success weighted by path length (SPL). SR measures the proportion of successful episodes, while SPL further penalizes successful episodes by the ratio of the optimal path length to the actual path length. Our evaluation protocol is consistent with prior work (Zhang et al., 2024a;

Zhu et al., 2025) and follows standard practice. Specifically, our simulation is configured with a 120° HFOV, a 1.5m sensor height, a 1.0m success distance threshold, and a 640×480 observation resolution.

While SPL is a standard metric for measuring path efficiency, it overlooks the computational cost incurred by reasoning models. To address this limitation, we introduce **Success weighted by Operation Time (SOT)**, a metric that penalizes successful episodes by the ratio of optimal search time to actual search time:

$$\text{SOT} := \frac{1}{N} \sum_{i=1}^N S_i \frac{T_{\text{optimal}}}{T_{\text{actual}}}.$$

Here, $S_i \in \{0, 1\}$ is the success indicator, T_{optimal} is the theoretical minimum physical time required to reach the target, while T_{actual} represents the total operating time, comprising two components: (i) *robot execution time*, computed based on execution time of meta actions (e.g., 1.0s for MoveAhead); (ii) *model inference latency*, the time required for the VLM to generate reasoning and action tokens. See Appendix B for more details.

4.2. The overall performance of HYDRA-NAV

The final performance of HYDRA-NAV-IRFT. Table 1 presents a comprehensive comparison between HYDRA-NAV and existing approaches. HYDRA-NAV-IRFT achieves new state-of-the-art (SOTA) results in SR and SPL across all settings, compared with methods that either output meta actions directly or rely on external tools for navigation. On the HM3D validation set, HYDRA-NAV-IRFT achieves an SR of 84.8%, outperforming the previous SOTA Uni-Navid (Zhang et al., 2024a) by +11.1%. On the MP3D benchmark, which features larger scenes and more diverse object categories, HYDRA-NAV-IRFT improves over the previous SOTA CogNav (Cao et al., 2025) by nearly +17.4%. The advantages of HYDRA-NAV-IRFT are most evident on the challenging OVON benchmark, designed to test the agent's generalization ability to synonyms and unseen object categories. On the Val-Synonyms split, HYDRA-NAV-IRFT shows a +15.8% improvement over the previous SOTA NavR1 (Liu et al., 2025a). On the Val-Unseen split, which strictly evaluates zero-shot generalization, HYDRA-NAV-IRFT achieves an SR of 66.3%, significantly outperforming the previous SOTA NavFoM (Zhang et al., 2025a) (45.2%) by +21.1%.

Effectiveness of the curriculum training pipeline. From Table 1, we can observe that HYDRA-NAV achieves higher SR and SPL with more training stages. In particular, HYDRA-NAV-Base at stage 1 is inferior to previous SOTA, while HYDRA-NAV-SFT, after equipped HYDRA-NAV with reasoning and memory, shows significant improvement (33.1% on the SR of HM3D) across all benchmarks. The

Table 1. Comparison with state-of-the-art methods on HM3D, MP3D, and OVON benchmarks. The attributes denote: **Low**: low-level action output, **High**: high-level planning/action output, **Dual**: dual-system architecture. **RGB**: uses RGB observations. **Depth**: uses Depth observations. The **best** and the second best results are denoted by **bold** and underline.

Method	Observation		Action		Dual	HM3D		MP3D		OVON					
	RGB	Depth	Low	High		Val		Val		Val-Seen		Val-Synonyms		Val-Unseen	
						SR	SPL	SR	SPL	SR	SPL	SR	SPL	SR	SPL
VoroNav (Wu et al., 2024)	✓	✓		✓		42.0	26.0	-	-	-	-	-	-	-	-
InstructNav (Long et al., 2024)	✓	✓		✓		50.0	20.9	-	-	-	-	-	-	-	-
VLMnav (Goetting et al., 2024)	✓	✓		✓		50.4	21.0	-	-	-	-	-	-	-	-
L3MVN (Yu et al., 2023)	✓	✓		✓		50.4	23.1	34.9	14.5	-	-	-	-	-	-
VLFM (Yokoyama et al., 2024a)	✓	✓		✓		52.5	30.4	36.4	17.5	35.2	18.6	32.4	17.3	35.2	19.6
GAMap (Yuan et al., 2024)	✓	✓		✓		53.1	26.0	-	-	-	-	-	-	-	-
SG-Nav (Yin et al., 2024)	✓	✓		✓		54.0	24.9	40.2	16.0	-	-	-	-	-	-
UniGoal (Yin et al., 2025)	✓	✓		✓		54.5	25.1	41.0	16.4	-	-	-	-	-	-
CompassNav (Li et al., 2025)	✓	✓		✓		56.6	27.6	42.0	17.5	-	-	-	-	43.5	21.6
BeliefMapNav (Zhou et al., 2025b)	✓	✓		✓		61.4	30.4	37.3	<u>17.6</u>	-	-	-	-	-	-
TriHelper (Zhang et al., 2024c)	✓	✓		✓		62.0	25.3	-	-	-	-	-	-	-	-
MTU3D (Zhu et al., 2025)	✓	✓		✓		-	-	-	-	55.0	23.6	45.0	14.7	40.8	12.1
WMNav (Nie et al., 2025)	✓	✓		✓		72.2	33.3	45.2	17.2	-	-	-	-	-	-
CogNav (Cao et al., 2025)	✓	✓		✓		72.5	26.2	46.6	16.1	-	-	-	-	-	-
NavFoM (Zhang et al., 2025a)	✓		✓			-	-	-	-	40.1	27.1	45.4	32.6	45.2	31.9
Nav- R^2 (Xiang et al., 2025)	✓		✓			-	-	-	-	45.6	21.0	45.9	21.1	44.0	18.0
Nav-R1 (Liu et al., 2025a)	✓	✓	✓	✓	✓	-	-	-	-	<u>58.4</u>	26.3	48.1	23.1	42.2	20.1
zson (Majumdar et al., 2022)	✓		✓			25.5	12.6	15.3	4.8	-	-	-	-	-	-
Navid (Zhang et al., 2024b)	✓		✓			32.5	21.6	-	-	-	-	-	-	-	-
PixNav (Cai et al., 2024)	✓		✓	✓	✓	37.9	20.5	-	-	-	-	-	-	-	-
ESC (Zhou et al., 2023)	✓	✓	✓	✓		39.2	22.3	28.7	11.2	-	-	-	-	-	-
Uni-Navid (Zhang et al., 2024a)	✓		✓			<u>73.7</u>	<u>37.1</u>	-	-	41.3	21.1	43.9	21.8	39.5	19.8
HYDRA-NAV-Base (Stage 1)	✓		✓	✓	✓	39.8	26.0	30.9	14.6	31.9	25.8	32.2	24.4	32.3	21.9
HYDRA-NAV-SFT (Stage 2)	✓		✓	✓	✓	72.9	27.7	<u>49.0</u>	16.5	57.2	<u>28.3</u>	<u>56.9</u>	27.2	<u>50.9</u>	23.1
HYDRA-NAV-IRFT (Stage 3)	✓		✓	✓	✓	84.8	41.1	64.0	29.6	65.0	34.4	63.9	35.1	66.3	37.4

performance of HYDRA-NAV-SFT is close to the previous SOTA on HM3D and achieves new SOTA on the MP3D and OVON-Val-Unseen. Furthermore, IRFT not only increases the search efficiency of HYDRA-NAV but also boosts the final success rate. Combining the efforts from all three stages, HYDRA-NAV-IRFT establishes highest SR and SPL across all settings, outperforming the second-best by a visible margin (+21.1% on the SR of OVON-Val-Unseen).

4.3. Ablation Studies

We perform an ablation study to analyze the contribution of different training strategies used in HYDRA-NAV. The results on HM3D Val and OVON Val-Unseen with different training setups are shown in Table 2.

Effectiveness of memory. Comparing Exp 1, 2, 3, and 7, we observe that the model without memory (Exp 1) achieves reasonable SR but suffers from low SPL compared to the other three. This gap in SPL indicates that the memory module is crucial for search efficiency. Among Exp 2, 3, and 7, models with different number of landmarks in memory do not show significant differences in SR or SPL. We ultimately set the maximum number of landmarks to 10 for stage 2 due to its superior performance on the OVON Val-Unseen.

Impact of data collection strategy. In Section 3.4, we introduced a “heuristic waypoint selection” strategy to generate exploration trajectories (Expl.), rather than relying

solely on the shortest path strategy (S.P.). Comparing Exp 6 with Exp 7, the model trained only on shortest paths (Exp 6) suffers significant degradation in SR, with a 25.4% drop on HM3D and a 10.5% drop on OVON Val-Unseen, compared to the model trained on trajectories with exploration (Exp 7). This indicates that exploration is a key capability for achieving high success rates. Interestingly, the model trained with shortest paths (Exp 6) achieves higher SPL compared to Exp 7. We attribute this to the fact that the model trained on shortest paths bypasses exploratory behavior, thereby achieving greater efficiency in straightforward scenarios where the target is in close proximity.

Importance of the progressive training pipeline. Our curriculum training pipeline consists of three distinct stages. Skipping stage 1 results in a noticeable performance drop in both SR and SPL (Exp 5 vs. Exp 7). Specifically, SR decreases by 4.9% on HM3D and 9.1% on OVON. Without the alignment between visual observations and meta-actions learned in stage 1, we observe the model sometimes fails to generate collision-free actions, causing it to get stuck more frequently, particularly in the unseen OVON environment.

Co-training improves performance Comparing Exp 4 with Exp 7, removing co-training (Exp 4) leads to a slight drop in SR and SPL (e.g., 69.1% vs. 72.9% on HM3D). This suggests that co-training with general VQA data helps the VLM maintain its general image understanding capabilities, avoid overfitting to the specific navigation data distribution,

Table 2. Model performance with different training recipes. We analyze the effectiveness of different data strategies, progressive training pipeline, and memory configurations on HM3D and OVON Val-Unseen benchmark. The **best** and the second best results are denoted by **bold** and underline. Memory($L = k$) represents the memory with a maximum number of k landmarks.

Ablation group	Exp #	Module configuration					HM3D Val		OVON Val-Unseen	
		Memory	Co-training	Stage 1	Expl.	S.P.	SR	SPL	SR	SPL
Memory	1	w/o Memory	✓	✓	✓		61.9	13.9	49.7	13.3
	2	Memory ($L = 5$)	✓	✓	✓		74.6	28.8	<u>50.0</u>	22.3
	3	Memory ($L = 15$)	✓	✓	✓		70.8	28.1	47.5	22.4
Co-training	4	Memory ($L = 10$)		✓	✓		69.1	25.9	48.2	22.3
Training Stage	5	Memory ($L = 10$)	✓		✓		68.0	24.6	41.8	14.1
Data Collection Strategy	6	Memory ($L = 10$)	✓	✓		✓	47.5	<u>28.7</u>	40.4	28.1
HYDRA-NAV stage 2	7	Memory ($L = 10$)	✓	✓	✓		<u>72.9</u>	27.7	50.9	<u>23.1</u>

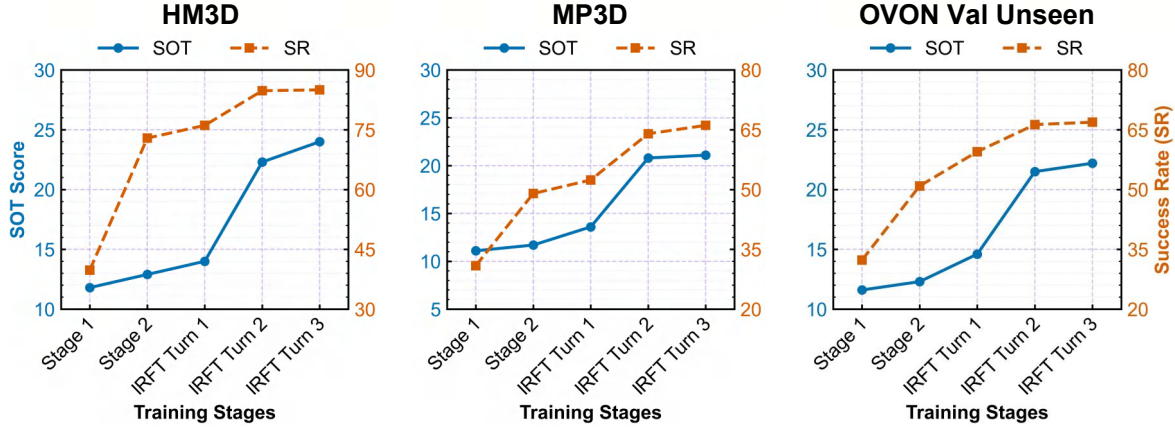


Figure 4. Performance analysis of multi-turn IRFT across different benchmarks.

and strengthen its ability to identify objects.

4.4. Effectiveness of Multi-turn IRFT

The evolution of SR and SOT across different training stages is presented in Figure 4. We observe that HYDRA-NAV achieves higher SR and SOT as training progresses. In particular, HYDRA-NAV-Base (stage 1) shows limited performance, while stage 2 yields a significant improvement in SR across all benchmarks (e.g., $\uparrow 33.1\%$ on HM3D). This substantial gain confirms that the base VLM benefits from the temporal-spatial reasoning introduced in Section 3.4, which enables the agent to handle long-horizon exploration and memory. However, during stage 2, we notice that the improvement in SOT is marginal compared to that in SR. This is because HYDRA-NAV-SFT (stage 2) operates with a fixed reasoning frequency and therefore suffers from high inference costs in simple scenarios where reactive control suffices. Following stage 2, IRFT not only boosts the final success rate but also dramatically increases search efficiency in the first two turns. As we carry out more iterations of RFT, HYDRA-NAV achieves new state-of-the-art results across all settings. In particular, on OVON Val-Unseen, HYDRA-NAV-IRFT improves SOT from 12.3% (stage 2) to 22.2% (RFT

turn 3). This demonstrates that the adaptive switching mechanism is effective: by learning from the “reject-and-repair” strategy, the agent learns to trigger the slow system only at critical “stagnation points” (e.g., lack of progress or repetitive exploration). This reduces unnecessary inference costs during the search process, resulting in a system that achieves both high SR and computational efficiency. We terminate IRFT at the third turn as we observe performance plateauing beyond this point. As detailed in Appendix D, compared to dense reasoning methods like VLMnav (Goetting et al., 2024) and Nav- R^2 (Xiang et al., 2025), HYDRA-NAV-IRFT achieves significantly higher SOT scores and SR by reducing the reasoning ratio to just 3.0%.

4.5. Real-world Deployment

We deploy HYDRA-NAV directly on a physical robot without real-world fine-tuning. The model demonstrate robust zero-shot transfer, successfully locating target objects within complex environments. See Appendix M for details.

5. Limitations

A key limitation of this work is that we evaluate our model exclusively in the Habitat simulator. This is primarily due to the lack of high-quality evaluation sets in more realistic simulators such as IsaacSim. Future work could develop new benchmarks in more realistic simulations and investigate navigation models that generalize across different simulators and embodiments.

6. Conclusion

In this paper, we present HYDRA-NAV, a navigation agent that balances success rate with execution efficiency by unifying reactive control and deliberate reasoning within a single VLM. Our dual-process architecture enhances spatio-temporal reasoning and long-term memory, enabling the agent to switch adaptively between fast execution and deliberate reasoning. To optimize this trade-off, we employ iterative rejection fine-tuning (IRFT), teaching the agent to trigger reasoning only at critical stagnation points. Extensive experiments on standard benchmarks demonstrate that HYDRA-NAV achieves state-of-the-art performance with superior efficiency. We hope our open-source code, models, and datasets will provide a robust foundation for future research into adaptive embodied agents.

Impact Statement

This work aims to advance the development of adaptive reasoning, vision-language model-based embodied agents for object navigation. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here. In addition, a discussion of possible future directions is provided in Appendix L.

Acknowledgements

This research was partially supported by the National Key Research and Development Program of China (Grant No. 2024YFB4505800), the Beijing Municipal Science and Technology Commission (Grant No. Z241100004224015).

References

- Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., Huang, F., Hui, B., Jiang, S., Li, Z., Li, M., Li, M., Li, K., Lin, Z., Lin, J., Liu, X., Liu, J., Liu, C., Liu, Y., Liu, D., Liu, S., Lu, D., Luo, R., Lv, C., Men, R., Meng, L., Ren, X., Ren, X., Song, S., Sun, Y., Tang, J., Tu, J., Wan, J., Wang, P., Wang, P., Wang, Q., Wang, Y., Xie, T., Xu, Y., Xu, H., Xu, J., Yang, Z., Yang, M., Yang, J., Yang, A., Yu, B., Zhang, F., Zhang, H., Zhang, X., Zheng, B., Zhong, H., Zhou, J., Zhou, F., Zhou, J., Zhu, Y., and Zhu, K. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Cai, W., Huang, S., Cheng, G., Long, Y., Gao, P., Sun, C., and Dong, H. Bridging zero-shot object navigation and foundation models through pixel-guided navigation skill. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5228–5234. IEEE, 2024.
- Cai, Y., He, X., Wang, M., Guo, H., Yau, W.-Y., and Lv, C. Cl-cotnav: Closed-loop hierarchical chain-of-thought for zero-shot object-goal navigation with vision-language models. *arXiv preprint arXiv:2504.09000*, 2025.
- Cao, Y., Zhang, J., Yu, Z., Liu, S., Qin, Z., Zou, Q., Du, B., and Xu, K. Cognav: Cognitive process modeling for object goal navigation with llms. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9550–9560, 2025.
- Chang, A., Dai, A., Funkhouser, T., Halber, M., Niessner, M., Savva, M., Song, S., Zeng, A., and Zhang, Y. Matterport3d: Learning from rgb-d data in indoor environments. *arXiv preprint arXiv:1709.06158*, 2017.
- Chaplot, D. S., Gandhi, D., Gupta, S., Gupta, A., and Salakhutdinov, R. Learning to explore using active neural slam. *arXiv preprint arXiv:2004.05155*, 2020a.
- Chaplot, D. S., Gandhi, D. P., Gupta, A., and Salakhutdinov, R. R. Object goal navigation using goal-oriented semantic exploration. *Advances in Neural Information Processing Systems*, 33:4247–4258, 2020b.
- Chaplot, D. S., Salakhutdinov, R., Gupta, A., and Gupta, S. Neural topological slam for visual navigation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 12875–12884, 2020c.
- Chen, S. and Li, B. Toward adaptive reasoning in large language models with thought rollback. *arXiv preprint arXiv:2412.19707*, 2024.
- Chung, S., Du, W., and Fu, J. Thinker: Learning to think fast and slow. *arXiv preprint arXiv:2505.21097*, 2025.
- Ding, X., Wei, J., Yang, Y., Jiang, S., Zhang, Q., Wu, H., Jia, F., Mi, L., Yan, Y., Wang, W., et al. Adanav: Adaptive reasoning with uncertainty for vision-language navigation. *arXiv preprint arXiv:2509.24387*, 2025.
- Druon, R., Yoshiyasu, Y., Kanazaki, A., and Watt, A. Visual object search by learning spatial context. *IEEE Robotics and Automation Letters*, 5(2):1279–1286, 2020.
- Gao, C., Jin, L., Peng, X., Zhang, J., Deng, Y., Li, A., Wang, H., and Liu, S. Octonav: Towards generalist embodied navigation. *arXiv preprint arXiv:2506.09839*, 2025.

- Goetting, D., Singh, H. G., and Loquercio, A. End-to-end navigation with vision language models: Transforming spatial reasoning into question-answering. *arXiv preprint arXiv:2411.05755*, 2024.
- Grandia, R., Jenelten, F., Yang, S., Farshidian, F., and Hutter, M. Perceptive locomotion through nonlinear model-predictive control. *IEEE Transactions on Robotics*, 39(5): 3402–3421, 2023.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Habibpour, M. and Afghah, F. History-augmented vision-language models for frontier-based zero-shot object navigation. *arXiv preprint arXiv:2506.16623*, 2025.
- Li, L., Zhao, J., Xie, Y., Tan, X., and Li, X. Compassnav: Steering from path imitation to decision understanding in navigation. *arXiv preprint arXiv:2510.10154*, 2025.
- Liu, Q., Huang, T., Zhang, Z., and Tang, H. Nav-r1: Reasoning and navigation in embodied scenes. *arXiv preprint arXiv:2509.10884*, 2025a.
- Liu, Y., Chen, W., Bai, Y., Liang, X., Li, G., Gao, W., and Lin, L. Aligning cyber space with physical world: A comprehensive survey on embodied ai. *IEEE/ASME Transactions on Mechatronics*, 2025b.
- Long, Y., Cai, W., Wang, H., Zhan, G., and Dong, H. Instructnav: Zero-shot system for generic instruction navigation in unexplored environment. *arXiv preprint arXiv:2406.04882*, 2024.
- Luo, Y., Li, F., Xu, S., Lai, Z., Yang, L., Chen, Q., Luo, Z., Xie, Z., Jiang, S., Liu, J., et al. Adathinkdrive: Adaptive thinking via reinforcement learning for autonomous driving. *arXiv preprint arXiv:2509.13769*, 2025.
- Lyu, Y., Shi, Y., and Zhang, X. Improving target-driven visual navigation with attention on 3d spatial relationships. *Neural Processing Letters*, 54(5):3979–3998, 2022.
- Majumdar, A., Aggarwal, G., Devnani, B., Hoffman, J., and Batra, D. Zson: Zero-shot object-goal navigation using multimodal goal embeddings. *Advances in Neural Information Processing Systems*, 35:32340–32352, 2022.
- Nasiriany, S., Xia, F., Yu, W., Xiao, T., Liang, J., Dasgupta, I., Xie, A., Driess, D., Wahid, A., Xu, Z., et al. Pivot: Iterative visual prompting elicits actionable knowledge for vlms. *arXiv preprint arXiv:2402.07872*, 2024.
- Nie, D., Guo, X., Duan, Y., Zhang, R., and Chen, L. Wmnav: Integrating vision-language models into world models for object goal navigation. *arXiv preprint arXiv:2503.02247*, 2025.
- Peng, Z., Ding, W., You, Y., Chen, Y., Luo, W., Tian, T., Cao, Y., Sharma, A., Xu, D., Ivanovic, B., et al. Counterfactual vla: Self-reflective vision-language-action model with adaptive reasoning. *arXiv preprint arXiv:2512.24426*, 2025.
- Qiao, Y., Lyu, W., Wang, H., Wang, Z., Li, Z., Zhang, Y., Tan, M., and Wu, Q. Open-nav: Exploring zero-shot vision-and-language navigation in continuous environment with open-source llms. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6710–6717. IEEE, 2025.
- Ramakrishnan, S. K., Chaplot, D. S., Al-Halah, Z., Malik, J., and Grauman, K. Poni: Potential functions for objectgoal navigation with interaction-free learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18890–18900, 2022.
- Sun, J., Wu, J., Ji, Z., and Lai, Y.-K. A survey of object goal navigation. *IEEE Transactions on Automation Science and Engineering*, 22:2292–2308, 2024.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Wang, S., Luo, Y., Chen, X., Luo, A., Li, D., Liu, C., Chen, S., Zhang, Y., and Yu, J. Vlingnav: Embodied navigation with adaptive reasoning and visual-assisted linguistic memory. *arXiv preprint arXiv:2601.08665*, 2026.
- Wang, X., Huang, Y., Wang, Y., Luo, X., Guo, K., Zhou, Y., and Zhang, X. Adareasoner: Adaptive reasoning enables more flexible thinking in large language models. *Preprint*, 2025a.
- Wang, Z., Li, X., Ye, Y., Fang, J., Wang, H., Liu, L., Liang, S., Lu, J., Wu, Z., Feng, J., et al. Game-tars: Pretrained foundation models for scalable generalist multimodal game agents. *arXiv preprint arXiv:2510.23691*, 2025b.
- Wei, M., Wan, C., Peng, J., Yu, X., Yang, Y., Feng, D., Cai, W., Zhu, C., Wang, T., Pang, J., et al. Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation. *arXiv preprint arXiv:2512.08186*, 2025.
- Wijmans, E., Kadian, A., Morcos, A., Lee, S., Essa, I., Parikh, D., Savva, M., and Batra, D. Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames. *arXiv preprint arXiv:1911.00357*, 2019.

- Wu, C., Li, B., Gao, M., and Wang, Z. From efficiency to adaptivity: A deeper look at adaptive reasoning in large language models. *arXiv preprint arXiv:2511.10788*, 2025a.
- Wu, P., Mu, Y., Wu, B., Hou, Y., Ma, J., Zhang, S., and Liu, C. Voronav: Voronoi-based zero-shot object navigation with large language model. *arXiv preprint arXiv:2401.02695*, 2024.
- Wu, S., Xie, J., Zhang, Y., Chen, A., Zhang, K., Su, Y., and Xiao, Y. Arm: Adaptive reasoning model. *arXiv preprint arXiv:2505.20258*, 2025b.
- Xiang, W., Zhang, H., Yang, T., Chu, Z., Chu, R., Xie, S., Yuan, Y., Sun, J., Gu, Z., Wang, J., et al. Nav- R^2 dual-relation reasoning for generalizable open-vocabulary object-goal navigation. *arXiv preprint arXiv:2512.02400*, 2025.
- Xue, X., Hu, J., Luo, M., Shichao, X., Chen, J., Xie, Z., Kuichen, Q., Wei, G., Xu, M., and Chu, Z. Omninav: A unified framework for prospective exploration and visual-language navigation. *arXiv preprint arXiv:2509.25687*, 2025.
- Yadav, K., Ramrakhya, R., Ramakrishnan, S. K., Gervet, T., Turner, J., Gokaslan, A., Maestre, N., Chang, A. X., Batra, D., Savva, M., et al. Habitat-matterport 3d semantics dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4927–4936, 2023.
- Yang, W., Wang, X., Farhadi, A., Gupta, A., and Mottaghi, R. Visual semantic navigation using scene priors. *arXiv preprint arXiv:1810.06543*, 2018.
- Yang, Y., Yang, H., Zhou, J., Chen, P., Zhang, H., Du, Y., and Gan, C. 3d-mem: 3d scene memory for embodied exploration and reasoning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 17294–17303, 2025.
- Yin, H., Xu, X., Wu, Z., Zhou, J., and Lu, J. Sg-nav: Online 3d scene graph prompting for llm-based zero-shot object navigation. *Advances in neural information processing systems*, 37:5285–5307, 2024.
- Yin, H., Xu, X., Zhao, L., Wang, Z., Zhou, J., and Lu, J. Unigoal: Towards universal zero-shot goal-oriented navigation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 19057–19066, 2025.
- Yokoyama, N., Ha, S., Batra, D., Wang, J., and Bucher, B. Vlfm: Vision-language frontier maps for zero-shot semantic navigation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 42–48. IEEE, 2024a.
- Yokoyama, N., Ramrakhya, R., Das, A., Batra, D., and Ha, S. Hm3d-ovon: A dataset and benchmark for open-vocabulary object goal navigation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5543–5550. IEEE, 2024b.
- Yu, B., Kasaei, H., and Cao, M. L3mvn: Leveraging large language models for visual target navigation. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3554–3560. IEEE, 2023.
- Yu, Z., Xu, T., Jin, D., Sankararaman, K. A., He, Y., Zhou, W., Zeng, Z., Helenowski, E., Zhu, C., Wang, S., et al. Think smarter not harder: Adaptive reasoning with inference aware optimization. *arXiv preprint arXiv:2501.17974*, 2025.
- Yuan, S., Huang, H., Hao, Y., Wen, C., Tzes, A., and Fang, Y. Gamap: Zero-shot object goal navigation with multi-scale geometric-affordance guidance. *Advances in Neural Information Processing Systems*, 37:39386–39408, 2024.
- Yue, J., Zhang, Y., Qin, C., Li, B., Lie, X., Yu, X., Zhang, W., and Zhao, Z. Think hierarchically, act dynamically: Hierarchical multi-modal fusion and reasoning for vision-and-language navigation. *arXiv preprint arXiv:2504.16516*, 2025.
- Zhan, Z., Deng, K., Tang, H., Xiang, W., Wu, K., Li, W., Zhu, W., Xu, J., Huang, L., Feng, Z., et al. Kat-vl: Kwai-autothink technical report. *arXiv preprint arXiv:2507.08297*, 2025.
- Zhang, J., Wang, K., Wang, S., Li, M., Liu, H., Wei, S., Wang, Z., Zhang, Z., and Wang, H. Uni-navid: A video-based vision-language-action model for unifying embodied navigation tasks. *arXiv preprint arXiv:2412.06224*, 2024a.
- Zhang, J., Wang, K., Xu, R., Zhou, G., Hong, Y., Fang, X., Wu, Q., Zhang, Z., and Wang, H. Navid: Video-based vlm plans the next step for vision-and-language navigation. *arXiv preprint arXiv:2402.15852*, 2024b.
- Zhang, J., Li, A., Qi, Y., Li, M., Liu, J., Wang, S., Liu, H., Zhou, G., Wu, Y., Li, X., et al. Embodied navigation foundation model. *arXiv preprint arXiv:2509.12129*, 2025a.
- Zhang, J., Lin, N., Hou, L., Feng, L., and Li, J. Adaptthink: Reasoning models can learn when to think. *arXiv preprint arXiv:2505.13417*, 2025b.
- Zhang, L., Zhang, Q., Wang, H., Xiao, E., Jiang, Z., Chen, H., and Xu, R. Trihelper: Zero-shot object navigation with dynamic assistance. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10035–10042. IEEE, 2024c.

Zhang, L., Hao, X., Tang, Y., Fu, H., Zheng, X., Wang, P., Wang, Z., Ding, W., and Zhang, S. NavA³: Understanding any instruction, navigating anywhere, finding anything. *arXiv preprint arXiv:2508.04598*, 2025c.

Zhong, Y., Feng, C., Yan, F., Liu, F., Zheng, L., and Ma, L. P3nav: A unified framework for embodied navigation integrating perception, planning, and prediction. *arXiv preprint arXiv:2503.18525*, 2025a.

Zhong, Y., Zhang, Z., Zhang, R., Huang, L., Gao, H., Wang, S., Li, D., Han, R., Guo, J., Peng, S., et al. Run, ruminate, and regulate: A dual-process thinking system for vision-and-language navigation. *arXiv preprint arXiv:2511.14131*, 2025b.

Zhou, K., Zheng, K., Pryor, C., Shen, Y., Jin, H., Getoor, L., and Wang, X. E. Esc: Exploration with soft common-sense constraints for zero-shot object navigation. In *International Conference on Machine Learning*, pp. 42829–42842. PMLR, 2023.

Zhou, X., Xiao, T., Liu, L., Wang, Y., Chen, M., Meng, X., Wang, X., Feng, W., Sui, W., and Su, Z. Fsr-vln: Fast and slow reasoning for vision-language navigation with hierarchical multi-modal scene graph. *arXiv preprint arXiv:2509.13733*, 2025a.

Zhou, Z., Hu, Y., Zhang, L., Li, Z., and Chen, S. Beliefmap-nav: 3d voxel-based belief map for zero-shot object navigation. *arXiv preprint arXiv:2506.06487*, 2025b.

Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., and Farhadi, A. Target-driven visual navigation in indoor scenes using deep reinforcement learning. In *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 3357–3364. IEEE, 2017.

Zhu, Z., Wang, X., Li, Y., Zhang, Z., Ma, X., Chen, Y., Jia, B., Liang, W., Yu, Q., Deng, Z., et al. Move to understand a 3d scene: Bridging visual grounding and exploration for efficient and versatile embodied navigation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 8120–8132, 2025.

Appendix

A. The definition of the score function

Below is the score function used in Section 3.4

$$S(p_{\text{init}}, p) := \underbrace{\frac{\min_{q \in U} \|p - q\|_2}{\max_{q \in U} \|p - q\|_2}}_{\text{spaciousness of } p} + \lambda \cdot \underbrace{\left(1 - \frac{\|p - p_{\text{target}}\|_2}{\max_{q \in G} (\|p_{\text{init}} - q\|_2)}\right)}_{\text{closeness to the target}},$$

where p represents the candidate point and p_{target} is the location of goal. U denotes the set of obstacle boundaries (e.g., walls). G represents the set of all targets in the scene. The first term calculates the spatial openness by measuring the distance from p to the nearest obstacle. The second term evaluates the proximity to the specific target, normalized by the distance from the start point p_{init} to the farthest target in G . The weight $\lambda = 0.7$ balances exploration against goal efficiency.

B. The computation of operation time

In this section, we provide the specific parameter settings and calculation method used for the Success weighted by Operation Time (SOT) metric. The total operating time T_{actual} consists of robot execution time and model inference latency, defined as follows:

$$T_{\text{actual}} = T_{\text{phys}} + T_{\text{inf}}$$

Physical Execution Estimation (T_{phys}). We calculate robot execution time by summing meta action durations. To ensure realism, these durations represent the average values obtained from extensive real-world testing on the physical robot, as listed in Table 3.

Table 3. Time cost parameters for robot physical actions.

Action Type	Time Cost (s)	Description
MoveAhead	1.0	Forward movement (approx. 0.25m)
RotateLeft/Right	0.6	In-place rotation (approx. 30°)
Obs	4.0	Panoramic observation
Stop	0.1	Signal to terminate the episode

Inference Latency Estimation (T_{inf}). The inference time is modeled as

$$T_{\text{inf}} = \tau \times (N_{\text{CoT}} + N_{\text{action}}),$$

where N_{CoT} and N_{action} represent the token counts for reasoning and actions. We set $\tau = 0.015s$ based on real-time inference logs collected during navigation on a single NVIDIA H20 GPU.

Theoretical minimum physical time (T_{optimal}). T_{optimal} represents the physical execution time required for the robot to traverse the shortest path in the scene.

B.1. Sensitivity Analysis of SOT to Inference Latency τ

To address concerns regarding the dependency of SOT on hardware configurations, we conduct a rigorous sensitivity analysis of the SOT score with respect to the per-token inference latency τ . We varied τ over a wide range, from 0.0075s (simulating GPUs like H100) to 0.48s (simulating high-latency edge scenarios). The results, summarized in Figure 5, demonstrate the robustness of our adaptive reasoning model through three key observations:

- **Consistent Superiority in Realistic Settings:** Across the realistic range of inference latencies ($\tau \in [0.0075, 0.12]$), HYDRA-NAV-IRFT (Turn 3) consistently achieves the highest SOT score. This indicates that our efficiency gains are not the result of a specific τ value but are intrinsic to the improved policy.
- **Resilience to High Latency via Adaptive Reasoning:** For models equipped with reasoning capability, IRFT proves highly resilient to hardware constraints. Comparing the adaptive reasoning model (IRFT) against the fixed-frequency reasoning model (stage 2), we observe that IRFT maintains a consistent upward trend in SOT across all latency

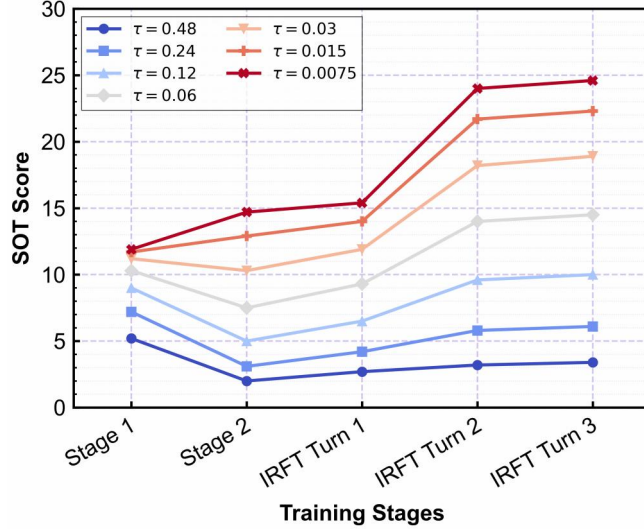


Figure 5. Performance visualization of HYDRA-NAV.

conditions. Notably, as hardware becomes slower (increasing τ), the performance gap widens in favor of IRFT. This confirms that by learning to trigger reasoning only at critical "stagnation points", IRFT maximizes the utility of compute, ensuring a net positive gain in efficiency even when inference costs are prohibitive.

- **Boundary Analysis:** The stage 1 model (which lacks reasoning) only begins to show competitive SOT at extremely high latencies ($\tau \geq 0.24$), solely due to its minimal inference overhead. However, within the practical operating range of modern VLMs, the navigational benefits of reasoning vastly outweigh the time cost, provided that the reasoning is triggered adaptively as done in HYDRA-NAV.

C. Training configurations

We use the AdamW optimizer ($\beta_1 = 0.9, \beta_2 = 0.95$) with bfloat16 precision and the cross-entropy loss for all three stages. A cosine learning rate scheduler is used with a warmup ratio of 0.1. We set the weight decay to 0.1 and gradient clipping to 1.0 for stability. The image inputs are fixed at 640×480 resolution with a max sequence length of 32768. Other training details are shown in Table 4.

Table 4. Hyperparameters for the three stage training pipeline.

Configuration	Stage 1	Stage 2	Stage 3
Dataset size	500k trajectories	565k mixed samples	~60k trajectories (per round)
Token count	20.1B	8.3B	4.5B
Base model	Qwen2.5-VL-7B	Stage 1 model	Stage 2 model
Hardware (NVIDIA H20)	128 GPUs	96 GPUs	64 GPUs
Global batch size	1024	768	512
Epochs	2	2	3
Training Cost	140 hours	100 hours	50 hours
Peak learning rate	1×10^{-5}		1×10^{-6}
Min learning rate	1×10^{-6}		1×10^{-7}

D. Comparison with SOTA baselines on SOT metric

D.1. Comparative SOT Results with SOTA Baselines

To evaluate the trade-off between success rate and computational overhead, we compare HYDRA-NAV with three representative open-source, reproducible baselines: VLMnav (Goetting et al., 2024), Nav- R^2 (Xiang et al., 2025), and WMNav (Nie

Table 5. Comparison with open-source, reproducible methods on HM3D and OVON Val-unseen benchmarks. The attributes denote: **Low**: low-level action output, **High**: high-level planning/action output. The **best** and the second best results are denoted by **bold** and underline.

Method	Action Level		HM3D Val		OVON Val-Unseen		Reasoning Ratio (%)	
	High	Low	SR $\uparrow$	SOT $\uparrow$	SR $\uparrow$	SOT $\uparrow$	HM3D	OVON Val-Unseen
VLMnav (Goetting et al., 2024)	✓		50.4	13.8	25.5	14.0	100.0	100.0
Nav- R^2 (Xiang et al., 2025)		✓	65.0	1.9	<u>44.0</u>	1.2	100.0	100.0
WMNav (Nie et al., 2025)	✓		<u>72.2</u>	<u>17.9</u>	33.4	<u>16.9</u>	100.0	100.0
HYDRA-NAV-IRFT Turn3		✓	84.3	24.0	66.3	20.3	3.0	3.5

et al., 2025). As shown in Table 5, we report the SR, SOT, and the reasoning ratio—defined as the percentage of steps where the reasoning is triggered.

Existing VLM-based methods typically rely on dense reasoning, where the reasoning is triggered at every step, resulting in a 100% reasoning ratio. Nav- R^2 , an end-to-end method that directly outputs low-level actions, achieves a competitive SR of 65.0% on HM3D. However, its SOT drops to just 1.9, primarily because triggering reasoning for every meta action causes significant time delays. This highlights that high success rates are insufficient for practical deployment if search efficiency is low. VLMnav and WMNav adopt a hierarchical framework approach to mitigate this, achieving better SOT scores (13.8 and 17.9 on HM3D). However, these hierarchical methods rely on the VLM acting as a brain to guide an ideal low-level executor. This dependency makes them difficult to deploy in the real world, where execution is imperfect.

In contrast, HYDRA-NAV-IRFT demonstrates that frequent reasoning is often unnecessary. By learning to trigger the reasoning only at critical “stagnation points” via IRFT, our model maintains an extremely low reasoning Ratio of 3.0% on HM3D and 3.5% on OVON Val-Unseen. Despite this sparse reasoning, HYDRA-NAV outperforms the second-best method (WMNav) by substantial margins: +12.1% SR on HM3D and +22.3% SR on OVON Val-Unseen. Consequently, HYDRA-NAV achieves the highest SOT scores across all benchmarks. This empirically validates that unifying reactive low-level control with adaptive, low-frequency reasoning is key to balancing high success rates with search efficiency, while its unified nature facilitates direct deployment in the real world.

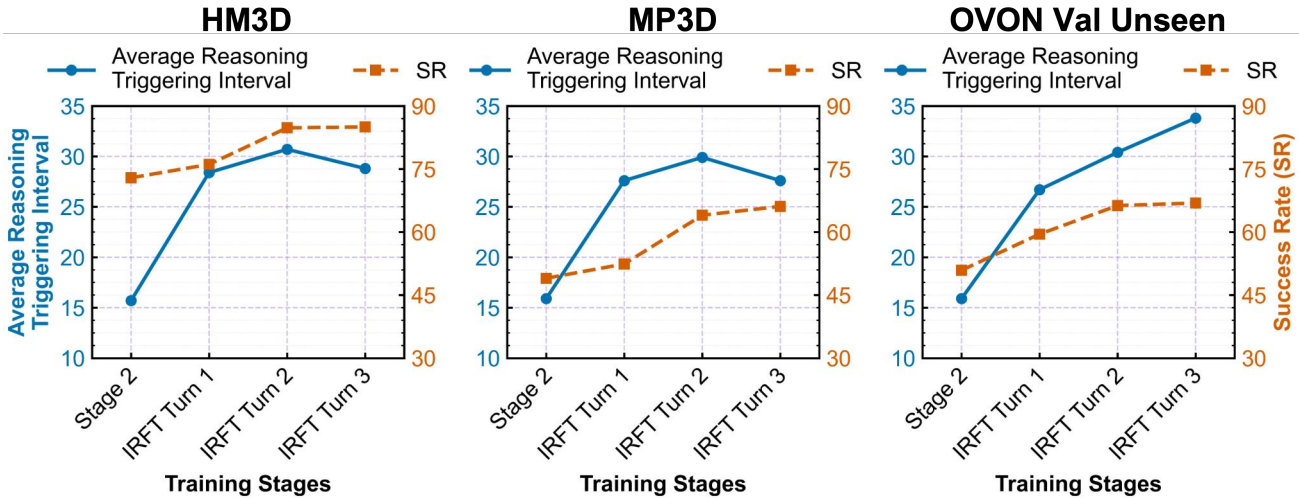


Figure 6. Reasoning Frequency analysis of multi-turn IRFT across different.

D.2. Analysis of Reasoning Frequency

Figure 6 shows the evolution of SR and reasoning frequency. Unlike stage 2, which relies on fixed, frequent reasoning (e.g., every 15.9 steps on OVON Val-Unseen), IRFT reduces the reasoning frequency while boosting performance. By Turn 3, the reasoning interval on OVON Val-Unseen doubles alongside a significant increase in SR by +16.0%. This confirms that the model learns to trigger the slow reasoning system only at critical “stagnation points.” We attribute the slight fluctuations in reasoning frequency to the fact that as the agent improves and survives longer, it encounters harder scenarios that require occasional reasoning. We terminate IRFT at turn 3 as the SR stabilizes.

E. Robustness and Ablation of Stage 3 Hyper-parameters

In this section, we discuss the robustness of the Stage 3 thresholds T_{stag} , δ_{stag} , and Δt , and whether optimal settings are consistent across varying scene scales, target densities, and benchmarks.

We observe these hyper-parameters to be highly consistent and robust. In our experiments, we employ the exact same parameters ($T_{stag} = 20$, $\Delta t \sim \mathcal{U}(20, 35)$, $\delta_{stag} = 0.5\text{m}$) across HM3D, MP3D, and OVON benchmarks. Table 6 presents an ablation study demonstrating that shifting these parameters barely affects the Success Rate (SR) or SPL.

Table 6. Ablation study of Stage 3 thresholds on OVON and MP3D benchmarks.

Ablation Variable	Setting	OVON				MP3D			
		SR	SPL	SOT	RR	SR	SPL	SOT	RR
Baseline (Ours)	$T_{stag} = 20, \Delta t \sim \mathcal{U}(20, 35), \delta_{stag} = 0.5\text{m}$	66.3	37.4	20.3	3.5	64.0	29.6	21.1	3.2
T_{stag}	$T_{stag} = 10$	65.5	36.7	19.1	4.1	63.2	28.9	19.7	3.8
	$T_{stag} = 30$	65.1	36.3	19.7	2.9	62.6	28.3	20.2	2.6
Δt	$\Delta t \sim \mathcal{U}(10, 20)$	64.5	35.7	18.0	4.4	62.1	27.9	18.6	4.1
	$\Delta t \sim \mathcal{U}(30, 50)$	63.2	34.6	18.8	2.5	60.4	26.8	19.5	2.3
δ_{stag}	$\delta_{stag} = 0.2\text{m}$	65.7	37.0	19.7	3.9	63.5	29.2	20.5	3.6
	$\delta_{stag} = 1.0\text{m}$	65.3	36.6	19.8	3.1	63.0	28.7	20.7	2.8

Furthermore, it is important to note that these hyper-parameters are only utilized to construct training data and are strictly irrelevant to model inference. During our development phase, we attempted to use Online Reinforcement Learning (such as GRPO) to let the model naturally learn when to trigger the slow system based on long-term efficiency rewards. However, we found that the performance improvements in Success Rate were marginal ($\pm 1\%$) compared to our Stage 2 model. Concurrently, training VLMs with online RL in embodied environments incurs massive computational costs. Therefore, we view the integration of stable, compute-efficient online RL for adaptive reasoning as a highly promising direction for future work.

F. Impact of Context Management and Data Scaling

It is important to note that HYDRA-NAV’s “dual-process architecture” does not introduce structural modifications to the underlying Vision-Language Model (VLM). Instead, it acts as a behavioral paradigm learned through our specific context management strategy, which interleaves memory, reasoning, and actions while dynamically switching policies.

To disentangle the performance gains yielded by our context management from those obtained by merely scaling up training data, we conduct an ablation study. We evaluate the Stage 2 model on the HM3D validation set using varying amounts of training data, as presented in Table 7.

Table 7. Data scaling ablation for the Stage 2 model on the HM3D validation set.

Data Volume	80k	160k	320k	460k	565k (Full)
HM3D SR (%)	56.2	63.8	68.2	70.4	72.9

The results indicate that the dual-process architecture drives the primary performance leap. While our purely reactive Stage 1 model achieves a 39.8% Success Rate (SR), integrating the dual-process architecture (Stage 2) instantly boosts the SR to 56.2% using merely 80k training samples. Further scaling the dataset to 565k pushes the SR to 72.9%. This ablation significantly highlights the remarkable sample efficiency of HYDRA-NAV: our method achieves state-of-the-art performance with approximately 565k samples, whereas several existing baselines rely on massive datasets ranging from 1.9M to 5.9M samples.

G. Impact of Base Model Capacity: Qwen2.5-VL vs. Qwen3-VL

In our primary experiments, we employ Qwen2.5-VL-7B as the backbone, as it is highly stable and fully optimized for our specific long-context, iterative rejection fine-tuning (IRFT) pipeline. To analyze whether the significant performance gains originate from the proposed framework, we additionally scale up our pipeline to support the stronger Qwen3-VL-8B model and train HYDRA-NAV across all three stages.

Table 8 summarizes the performance comparison between the two backbones. The metrics are reported as Success Rate (SR) and Success weighted by Path Length (SPL).

Table 8. Performance comparison of HYDRA-NAV using Qwen2.5-VL-7B and Qwen3-VL-8B across different training stages. Metrics are formatted as SR / SPL.

Training Stage	Backbone	HM3D Val	MP3D Val	OVON Val-Unseen
Stage 1	Qwen2.5-VL-7B	39.8 / 26.0	30.9 / 14.6	32.3 / 21.9
	Qwen3-VL-8B	48.5 / 28.0	39.2 / 17.3	41.5 / 23.3
Stage 2	Qwen2.5-VL-7B	72.9 / 27.7	49.0 / 16.5	50.9 / 23.1
	Qwen3-VL-8B	74.2 / 28.9	50.8 / 18.4	52.5 / 24.5
Stage 3	Qwen2.5-VL-7B	84.8 / 41.1	64.0 / 29.6	66.3 / 37.4
	Qwen3-VL-8B	85.7 / 41.6	65.1 / 30.2	65.9 / 37.3

The results clearly demonstrate that our proposed framework dictates the performance ceiling. In the purely reactive setup (Stage 1), upgrading to a structurally stronger backbone (Qwen3-VL-8B) naturally improves the baseline performance (e.g., boosting HM3D SR from 39.8% to 48.5%). However, once we introduce our dual-process architecture (Stage 2) and apply the full IRFT framework (Stage 3), both models converge to nearly identical state-of-the-art performance (e.g., 84.8% vs. 85.7% SR on HM3D). This indicates that the massive performance leap is driven by the adaptive reasoning framework doing the heavy lifting, rather than merely relying on the capability of the underlying Vision-Language Model.

H. Example of the long-term memory

The long-term memory consists the following information:

- **Landmark nodes:** Represented by the panoramic observations (e.g., “At landmark1...”), where `<image>` tokens correspond to the 4 RGB images obtained by rotating the head camera at 90° intervals (0°, 90°, 180°, 270°).
- **Action edges:** Represented by the execution meta actions connecting two consecutive landmarks.

Memory Example

```
At landmark1, you see <image><image><image><image>;
Executed RotateRight 30.0, MoveAhead 0.25, RotateRight 30.0, RotateRight 30.0,
MoveAhead 0.25, RotateLeft 30.0, MoveAhead 0.25, MoveAhead 0.25, MoveAhead 0.25,
RotateLeft 30.0, MoveAhead 0.25, RotateLeft 30.0, MoveAhead 0.25, MoveAhead 0.25,
MoveAhead 0.25, MoveAhead 0.25, MoveAhead 0.25, MoveAhead 0.25, MoveAhead 0.25,
MoveAhead 0.25, MoveAhead 0.25, MoveAhead 0.25, MoveAhead 0.25, RotateLeft 30.0,
RotateLeft 30.0, RotateLeft 30.0, RotateLeft 30.0 from landmark 1 to
landmark 2;
At landmark2, you see <image><image><image><image>;
Your current view is <image>.
```

I. Prompt for reasoning synthesis

We adopt the following two steps to generate reasoning text.

- Step 1: We prompt Qwen3-VL-235B-Thinking to summarize past visual observation along with the long-term memory.
- Step 2: Then we feed the summary from step 1, the current view, and the “future correct view” into the model. The model then generates reasoning text that serves to guide the future search plan.

The memory summary created in stage 1 is used as a key input for stage 2 to ensure the planning is based on history.

Step 1: Summarize History

1. Task Definition

- You are an autonomous navigation agent.
- Your task is to analyze your visual history and raw memory log to consolidate your exploration history into a concise summary. You must verify which areas you have already explored.

2. Inputs

- **Visual Stream:** [History_Img_1], ..., [History_Img_N]
(A full sequence of panoramic views from all previously visited landmarks).
- **Context:** You are provided with {number of images} images representing these views.
- **Raw Memory Log:** “{action edges}”

3. Output Format

- Output only the summary paragraph.
- Do not include introductory phrases like “Here is the summary”.

Step 2: Visual Hindsight Planning

1. Task Definition

- **Role:** Navigation Agent.
- **Goal:** {goal}
- **Objective:** Identify actionable visual cues (e.g., doorways, corridors, objects) in the current view that logically lead to the scene shown in the future view. Formulate a high-level plan based on these cues and your memory.

2. Inputs

- **Current View (Images 1-4):** [Img_1], [Img_2], [Img_3], [Img_4]
(A 360-degree panoramic observation of your current location).
- **Future View (Image 5):** [Image_5]
(The Ground Truth observation of the next step / Implicit Guide).
- **Memory Context:** “{memory_summary}”
(Consolidated history from Step 1).

3. Output Format

- Output the planning analysis and decision strictly in the following structure:
- **Visual Analysis:** “Analyzing the current view, I see...”
- **High-level Plan:** “Given that I have finished exploring [Area from Memory], and seeing [Feature] in the current view which likely leads to [Oracle Direction], I will proceed to...”

4. Rules & Constraints

- **Hindsight Mechanism:** Image 5 reveals the correct future path.
- **Critical Constraint:** You must pretend you have never seen Image 5. You must rationalize your decision based **only** on the Current View (Images 1-4) and your memory, using Image 5 only as a hidden ground-truth guide.
- **Sequential Dependency:** You must explicitly reference the provided memory context to justify why you are choosing a new direction versus backtracking (e.g., confirming previous areas are fully explored).

J. System prompts

We present the system prompts used for inference.

System prompt for HYDRA-NAV-IRFT (stage 3)

ROLE & MISSION

You are an expert autonomous navigation agent. You operate as a dual-process “fast and slow” thinking system to reach a user-specified goal.

“Slow System” (Planning): Activated at the start of the episode or following a self-initiated obs action. You will receive your full memory, the goal, and the current_view. You must analyze this information to create a high-level plan.

“Fast System” (Execution): You execute your plan by generating low-level navigation actions based solely on your current_view. You must remain in this mode until you reach the goal, determine a need for re-planning, OR reach the maximum execution limit (30 steps) since the last observation.

ACTION SPACE

You must choose only one action from this list per turn. Do not output any text other than the single chosen action.

- RotateLeft 30.0: Rotate 30 degrees to the left.
- RotateRight 30.0: Rotate 30 degrees to the right.
- MoveAhead 0.25: Move forward 0.25 meters.
- obs: Stop and perform a 360-degree observation. This action reactivates the “Slow System” to update your memory and formulate a new plan.
- end: Terminate the mission when you are confident that you are within 1.0 meter of the goal.

CONVERSATION FLOW & BEHAVIOR

1. THE “SLOW” SYSTEM (Planning Mode)

- Input: Goal, Memory, and Panoramic View.
- Output Format:


```
<think_start>
Summarize memory...
Analyze panoramic view...
Formulate plan...
<think_end>
[Action]
```

2. THE “FAST” SYSTEM (Execution Mode)

- Input: only the current_view.
- Output Format:


```
[Action]
```

CRITICAL INSTRUCTIONS

- Task Completion: You must output end exclusively when you are confident you are within 1.0 meter of the goal.
- Adaptive Observation: You should output obs if your path is blocked, the environment is ambiguous, or you are lost.
- Forced Observation: You must output obs if you have executed 30 consecutive actions in the “Fast System” without observing. The maximum interval between obs actions is strictly 35 steps.

K. Examples of trajectory**Instruction: Find a Sofa**

Figure 7. Performance visualization of HYDRA-NAV-Base.

Instruction: Find a Bed

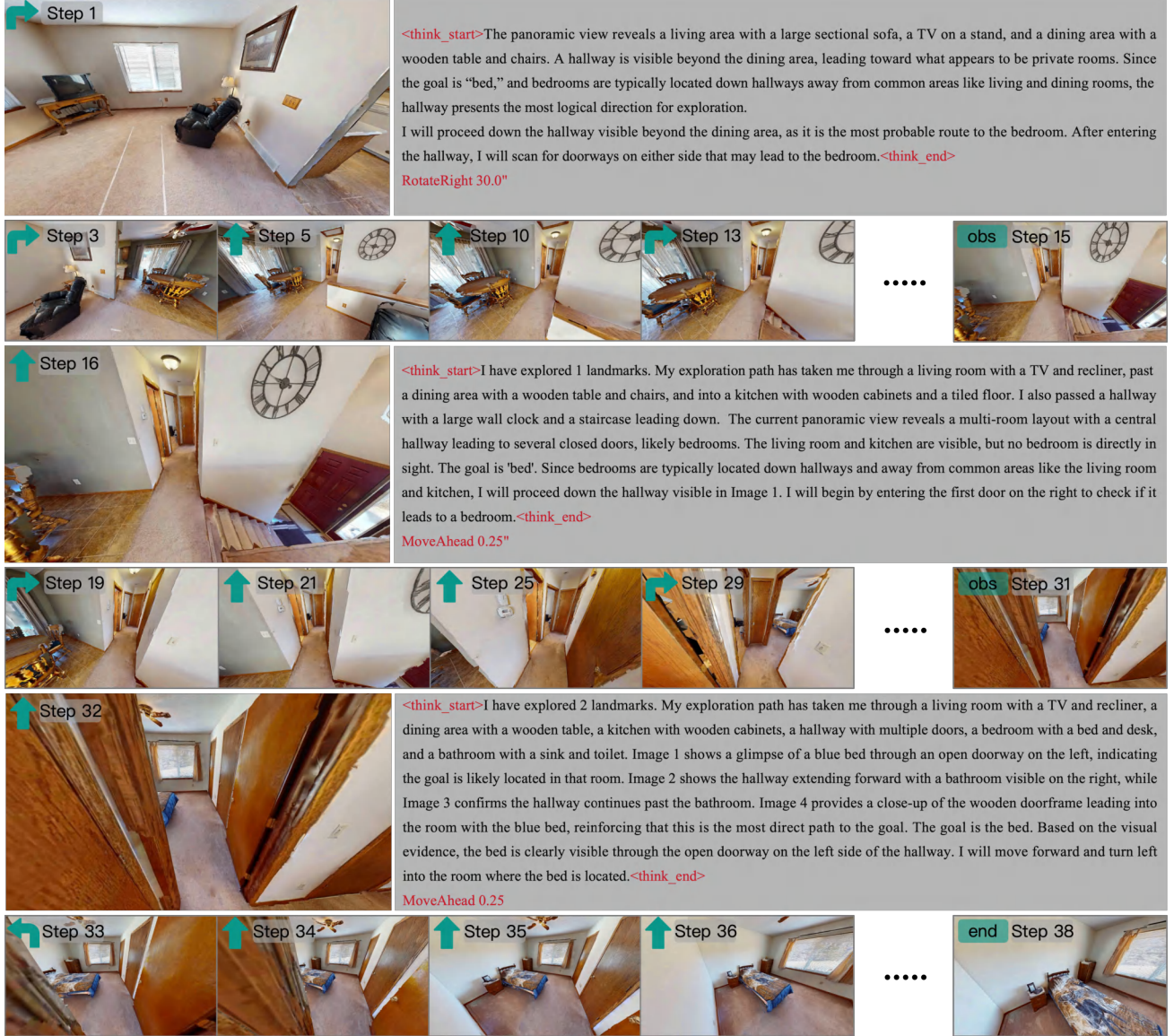


Figure 8. Performance visualization of HYDRA-NAV-SFT.

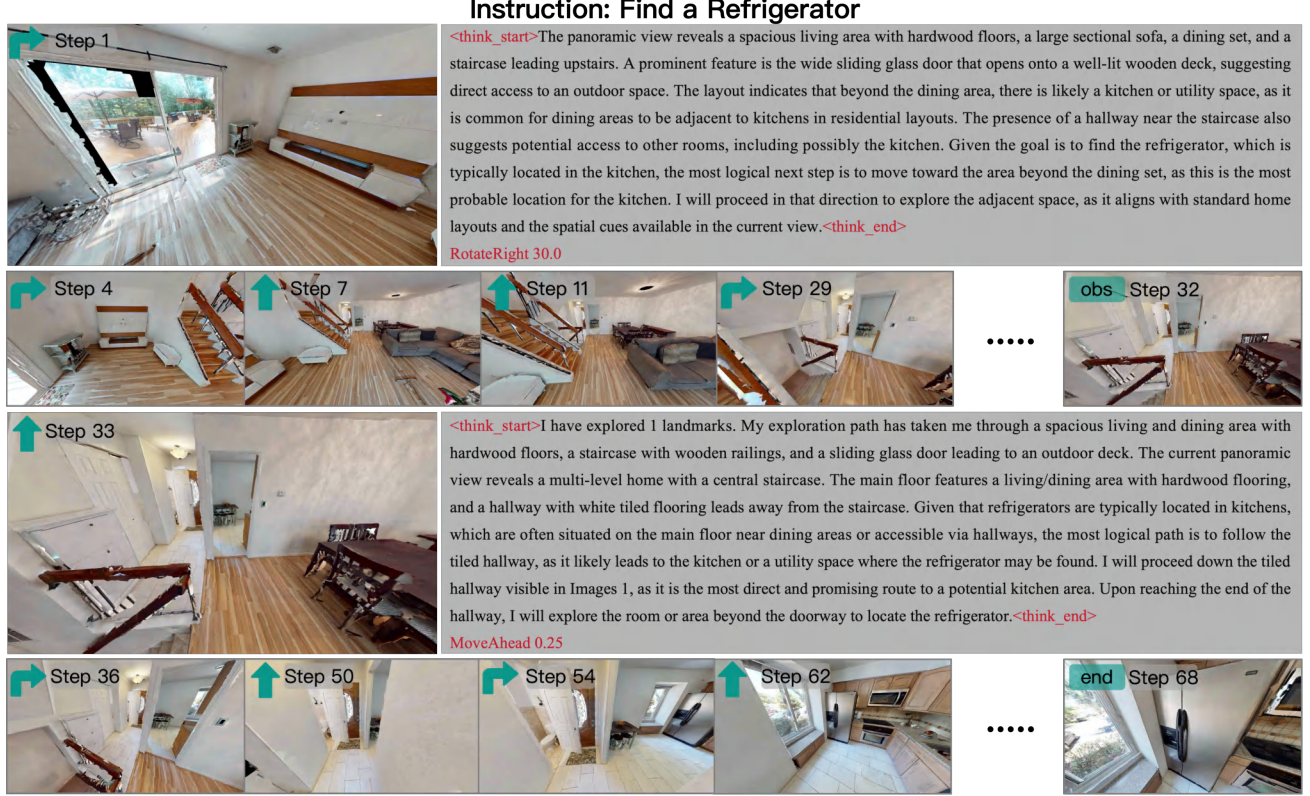


Figure 9. Performance visualization of HYDRA-NAV-IRFT.

L. Future Research Directions

While HYDRA-NAV represents a significant step forward in unifying reactive control and deliberate reasoning for object navigation, several challenges remain. We outline three promising directions for future research:

- **Generalization to Realistic Environments.** A key limitation of this work is that evaluations are conducted exclusively in the Habitat simulator due to the lack of high-quality benchmarks in more realistic environments like Isaac Sim. Future research should focus on developing high-fidelity benchmarks and investigating how well navigation models generalize across different simulators and, crucially, to the physical real world.
- **Optimizing Adaptive Switching Mechanisms.** While Iterative Rejection Fine-Tuning (IRFT) successfully teaches agents to reason at "stagnation points", future work could leverage online reinforcement learning. This would allow agents to autonomously learn the optimal timing for triggering reasoning and simultaneously improve the quality of the generated reasoning chains.
- **Expanding Embodiment and Task Scope.** Currently, HYDRA-NAV is tailored for object navigation. Future research could extend this dual-process architecture to a broader range of embodied tasks and robot types. This includes exploring how the "slow-fast" paradigm applies to more complex scenarios, such as mobile manipulation or autonomous game agents.

By exploring these directions, the field can move closer to developing intelligent embodied agents that effectively balance deliberate reasoning with execution efficiency in complex, real-world settings.

M. Robot Platform and Deployments

We visualize our robot platform in Figure 10. The system is based on a Unitree Go2 quadruped, equipped with an Intel RealSense D457 camera (utilizing 1280×800 RGB frames with a 90° HFOV) and a portable Wi-Fi module for

communication with a remote server (NVIDIA RTX 4090). During deployment, the robot transmits compressed images (640×480) to the server for next action prediction. The robot executes the predicted action using a nonlinear model predictive control (NMPC) module (Grandia et al., 2023), which computes optimal velocities based on a kinematic unicycle model. Figure 11 visualizes real-world navigation episodes where the robot starts from an initial position and successfully locates a box, trash can, and oven.



Figure 10. Real-world robot platform.

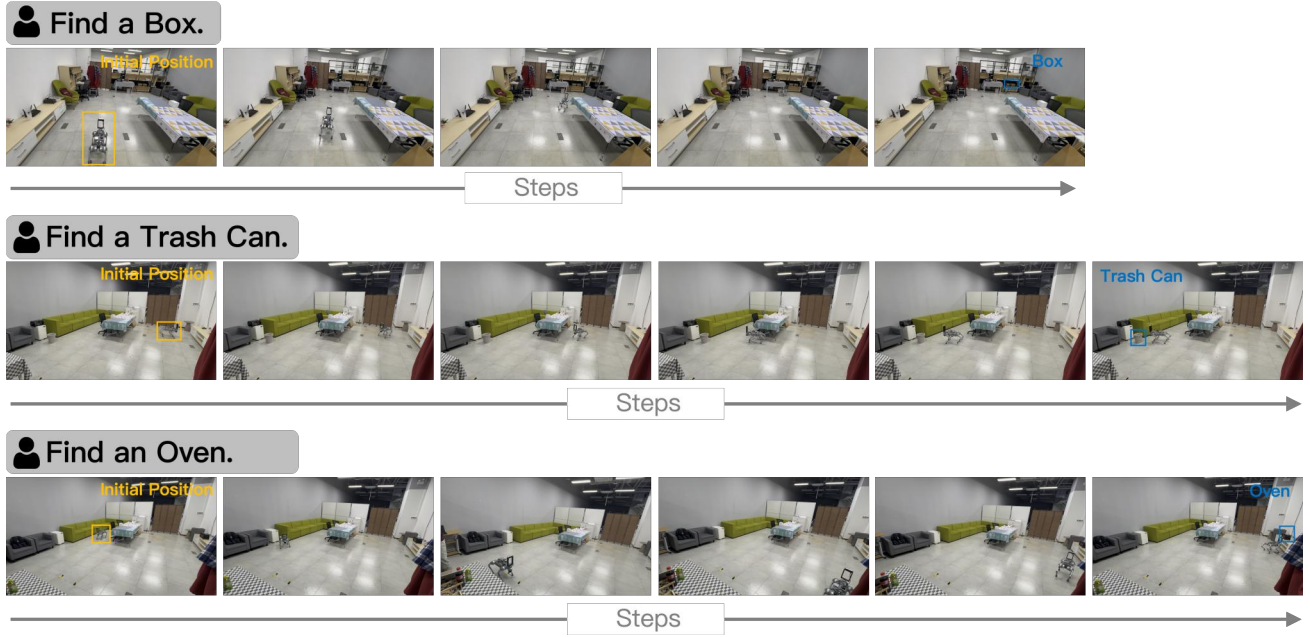


Figure 11. Qualitative performance of HYDRA-NAV in real-world deployments.

N. Data collection for Stage 3

Algorithm 1 Rollouts with Stagnation Point Detection

```

1: Input: Goal object set  $G$ , Initial state  $p_1$ , Max steps
    $T_{\max} = 200, T_{\text{stag}} = 20, \delta_{\text{stag}} = 0.5m$ 
2: Output: Trajectory  $\tau$ 
3: Initialize  $\tau \leftarrow \emptyset, t \leftarrow 1$ 
4:  $\text{ForceSlow} \leftarrow \text{True}$  // Start with slow system
5: while  $t \leq T_{\max}$  AND (not  $a_t = \text{End}$ ) do
6:   // 1. Check for stagnation point if in fast system
7:   if  $\text{ForceSlow} = \text{False}$  then
8:     Condition 1 (Repetitive exploration): Check if
        $\exists k \in [0, t - T_{\text{stag}}]$  s.t.  $\|p_t - p_k\| \leq \delta_{\text{stag}}$ 
9:     Condition 2 (Lack of progress): Sample  $\Delta t \sim \mathcal{U}(20, 35)$ . Check if  $\text{dist}(p_t, G) > \text{dist}(p_{t-\Delta t}, G)$ 
10:    if Condition 1 is Met OR Condition 2 is Met then
11:       $\text{ForceSlow} \leftarrow \text{True}$  // Trigger reasoning
        (slow) system intervention
12:    end if
13:  end if
14:
15:  // 2. Execute policy based on system mode
16:  if  $\text{ForceSlow} = \text{True}$  then
17:    // Slow system: reasoning + action
18:    Input "obs" token to trigger reasoning
19:    Generate reasoning
20:    Sample action  $a_t$  based on reasoning
21:     $\text{ForceSlow} \leftarrow \text{False}$  // Switch back to fast system
      immediately
22:  else
23:    // Fast system: reactive meta action
24:    Sample action  $a_t$  directly from policy
25:  end if
26:
27:  // 3. Environment step
28:  Execute  $a_t$ , observe  $p_{t+1}$ 
29:   $\tau \leftarrow \tau \cup \{(p_t, a_t)\}$ 
30:   $t \leftarrow t + 1$ 
31: end while
32: Return  $\tau$ 

```

Algorithm 2 IRFT Data Generation (Collection & Repair)

```

Input: Number of rollouts  $N$ , Goal object set  $G$ , Max
steps  $T_{\max} = 200, \delta_{\text{success}} = 1.0m$ 
2: Output: Training Dataset  $\mathcal{D}$ 
   // Phase 1: collect raw trajectories
4: Initialize raw dataset  $\mathcal{T}_{\text{raw}} \leftarrow \emptyset$ 
   for  $i = 1$  to  $N$  do
6:   Run Algorithm 1 to collect trajectory  $\tau$ 
    $\mathcal{T}_{\text{raw}} \leftarrow \mathcal{T}_{\text{raw}} \cup \{\tau\}$ 
8: end for

10: // Phase 2: process and repair
   Initialize final training dataset  $\mathcal{D} \leftarrow \emptyset$ 
12: while  $\mathcal{T}_{\text{raw}} \neq \emptyset$  do
   Pop a trajectory  $\tau$  from  $\mathcal{T}_{\text{raw}}$ 
14:   // Check if the raw trajectory is successful
   if  $a_T = \text{End} \wedge \text{dist}(p_T, G) \leq \delta_{\text{success}}$  then
16:      $\mathcal{D} \leftarrow \mathcal{D} \cup \{\tau\}$ 
   else
18:     // Trajectory failed: identify intervention time  $t^*$ 
     if  $a_T = \text{End} \wedge \text{dist}(p_T, G) > \delta_{\text{success}}$  then
20:       // Case 1: target misidentification
        $t^* \leftarrow T$ 
22:     else
       // Case 2: timeout
24:       Identify stagnation points  $S \subset [1, T]$ 
        $t^* \leftarrow \text{argmin}_{t \in S} \text{dist}(p_t, G)$ 
26:     end if

28:     // Perform repair strategy from  $t^*$ 
     Set action  $a_{t^*} \leftarrow \text{obs}$ 
30:     Generate oracle path  $\tau_{\text{oracle}}$  from  $p_{t^*}$  to  $G$  via  $A^*$ 
     Synthesize reasoning for  $\tau_{\text{oracle}}$ 
32:      $\tau_{\text{new}} \leftarrow \text{Concat}(\tau[1 : t^*], \tau_{\text{oracle}})$ 
     // Check if repaired trajectory is successful
34:     if  $|\tau_{\text{new}}| \leq 400$  AND  $\text{dist}(p_T, G) \leq \delta_{\text{success}}$  then
        $\mathcal{D} \leftarrow \mathcal{D} \cup \{\tau_{\text{new}}\}$ 
36:     end if
   end if
38: end while
Return  $\mathcal{D}$ 

```
